01

Analysis & Prediction of Property Valuation - Advanced Regression

California Housing Market

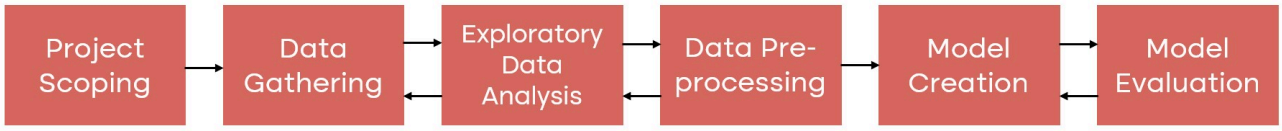
AJ

Aashish Joshi

Table of Contents

Research Environment

Data Science Workflow



---

1. Project Scope

Overview:

House prices tend to fluctuate every year, so there is a need for a system to predict house prices in the future. House price prediction can help the developer determine the selling price of a house and can help the customer to arrange the right time to purchase a house.

Finalizing scope:

We intend to predict the median monetary value of house prices in the districts of California based on leveraging the data science pipeline mentioned above.

Data requirements:

We will leverage the California Housing dataset obtained from the StatLib repository.

2. Data Gathering

In [1]:
```
#Importing required libraries
import numpy as np
import pandas as pd
```

In [2]:
```
#Importing the Dataset
from sklearn.datasets import fetch_california_housing
data = fetch_california_housing(as_frame=True)
dataset = data.frame
```

3. Exploratory Data Analysis

In [3]:
```
#Importing required libraries
import matplotlib.pyplot as plt
import seaborn as sns
```

In [4]:
```
#Quick data check
dataset.head()
```

Out[4]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

In [5]:
```
#Checking datatypes
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   MedInc       20640 non-null  float64
 1   HouseAge     20640 non-null  float64
 2   AveRooms     20640 non-null  float64
 3   AveBedrms    20640 non-null  float64
 4   Population   20640 non-null  float64
 5   AveOccup     20640 non-null  float64
 6   Latitude     20640 non-null  float64
 7   Longitude    20640 non-null  float64
 8   MedHouseVal  20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
```

All columns are numeric values (with the datatype float64)

Attribute Metadata:

1. MedInc: median income in block group

2. HouseAge: median house age in block group

3. AveRooms: average number of rooms per household

4. AveBedrms: average number of bedrooms per household

5. Population: block group population

6. AveOccup: average number of household members

7. Latitude: block group latitude

8. Longitude: block group longitude

9. MedHouseVal: The target variable is the median house value for California districts, expressed in hundreds of thousands of dollars ($100,000).

In [6]:
```
#Identifying missing data
dataset.isna().sum()
```

Out[6]:
```
MedInc         0
HouseAge       0
AveRooms       0
AveBedrms      0
Population     0
AveOccup       0
Latitude       0
Longitude      0
MedHouseVal    0
dtype: int64
```

No Missing Data

In [7]:
```
#Identifying duplicate rows
dataset[dataset.duplicated()]
```

Out[7]:

MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal

No Duplicate Data

In [8]:
```
#Running descriptive statistics
dataset.describe()
```

Out[8]:

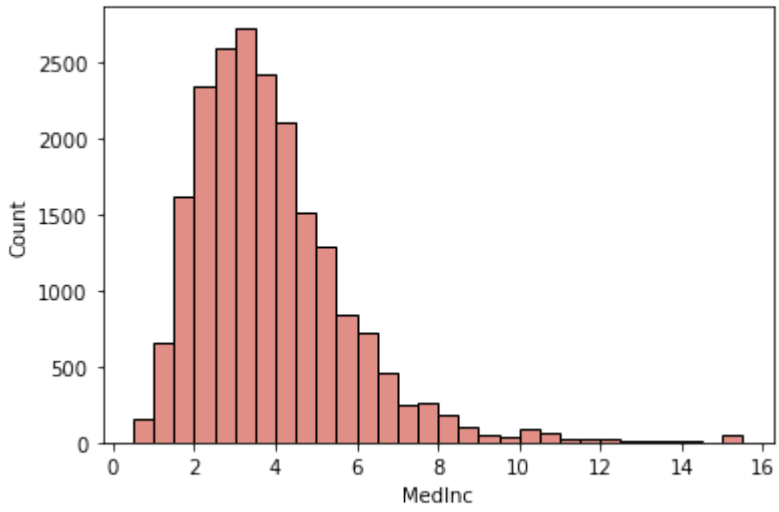
	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	L
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	2064
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-1
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-12
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429741	33.930000	-12
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-1
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-1
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-1

AveRooms, AveBedrms, Population and AveOccup columns all have a huge difference between their mean and maximum values.

This indicates the presence of outliers in the data.

Outlier detection :

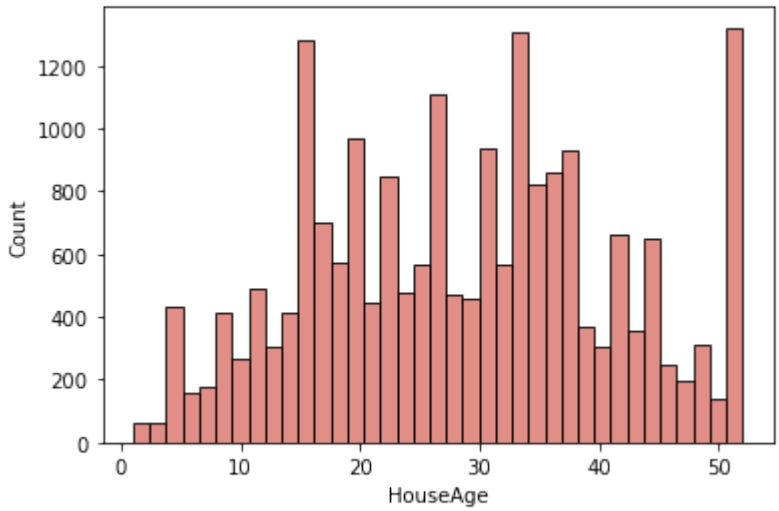
In [10]:
```
#Histogram for MedInc data
sns.histplot(dataset.MedInc, color='#D7685F', binwidth=0.5);
```



No observations in the median income data can be considered as outliers.

The histogram predominantly resembles a normal distribution with a small segment of people earning a substantially large income (which is typically what we expect with income related data)

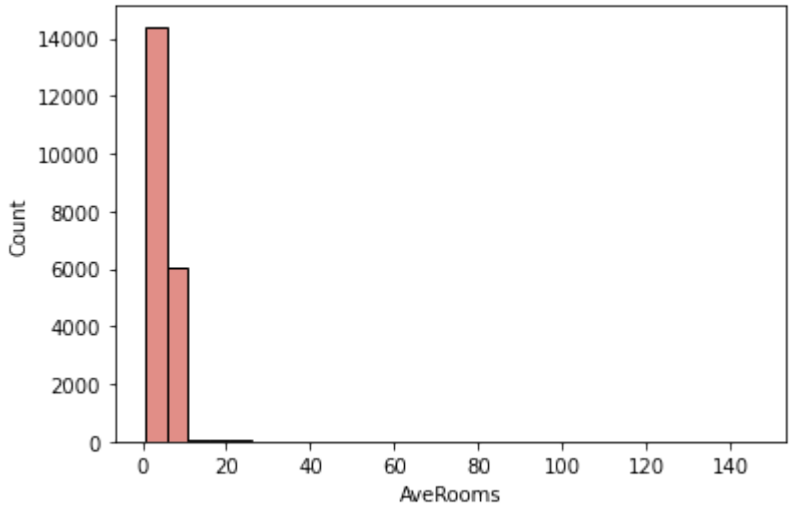
In [11]:
```
#Histogram for HouseAge data
sns.histplot(dataset.HouseAge, color='#D7685F');
```



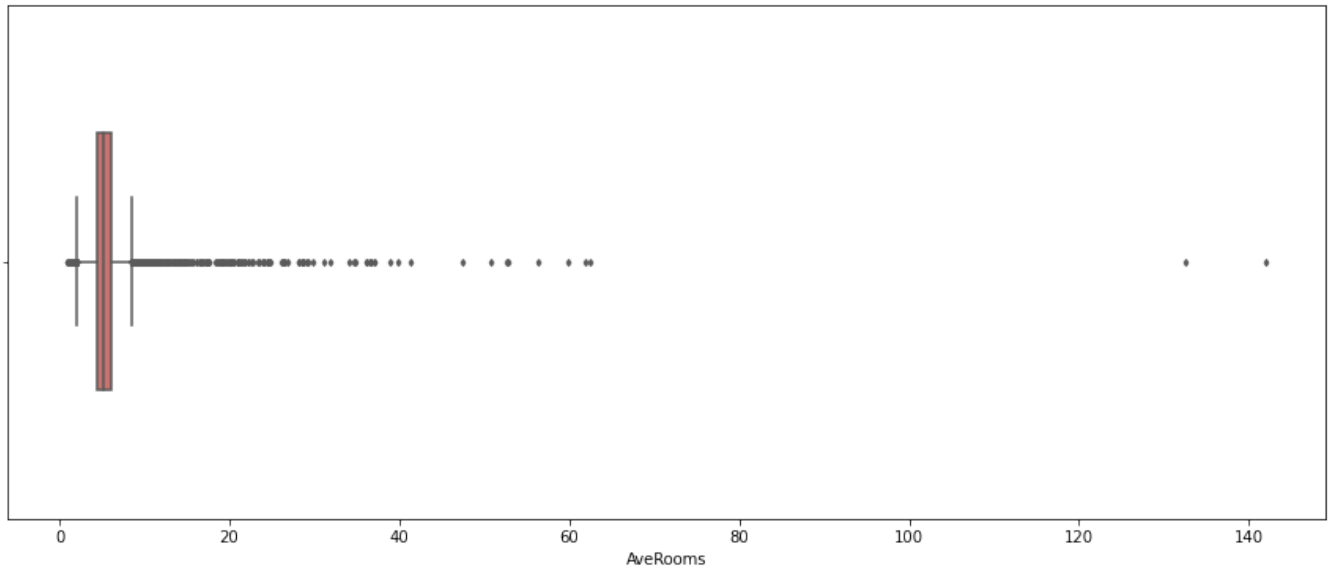
There are no discernable outliers with resepect to the age of the property.

The histogram largely resembles a normal distribution with no extreme values.

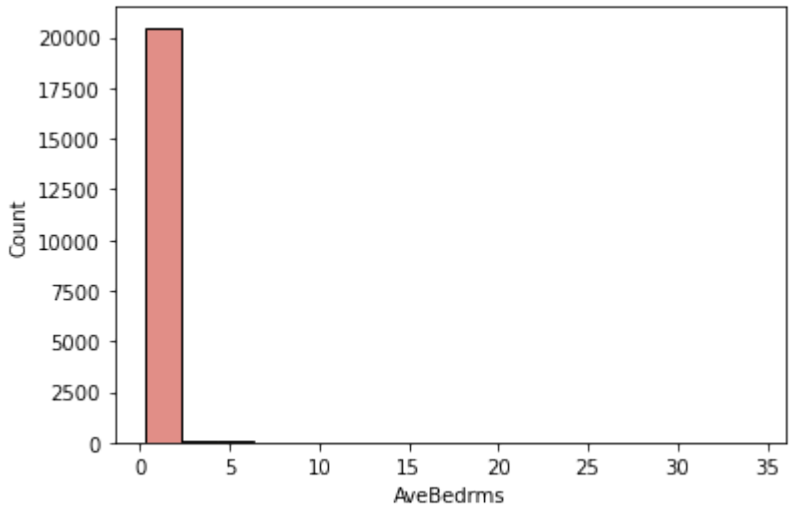
In [12]:
```
#Histogram for AveRooms data
sns.histplot(dataset.AveRooms, color='#D7685F',binwidth=5);
```



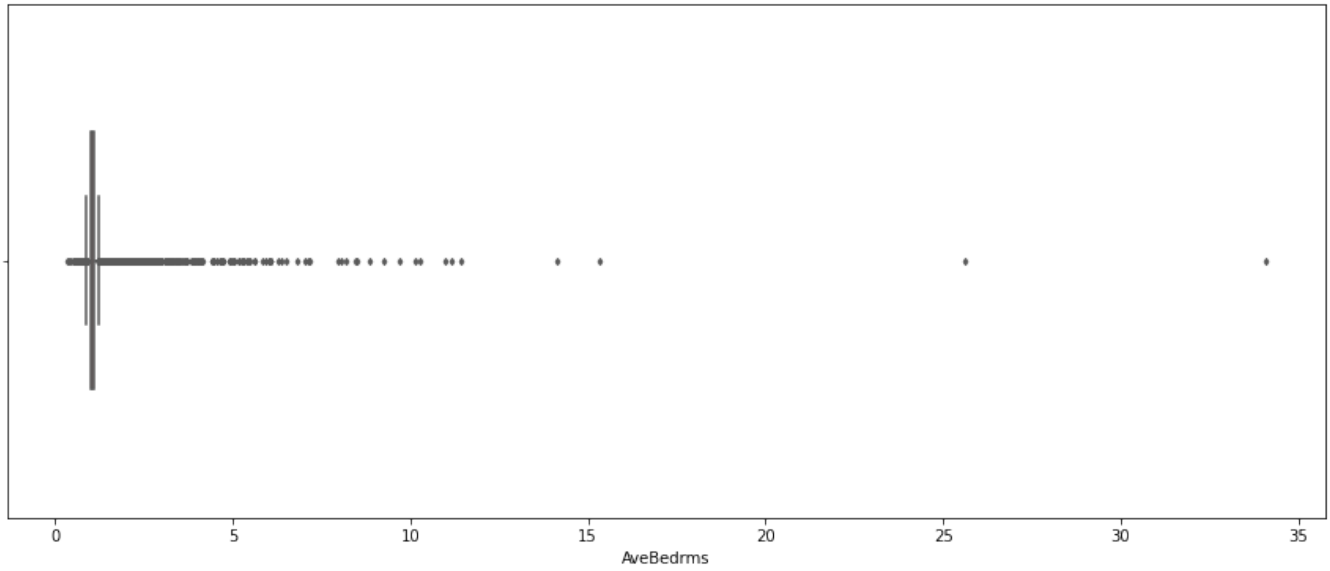
In [13]:
```
#Boxplot for AveRooms
plt.figure(figsize=(15, 6))
sns.boxplot(x=dataset.AveRooms, width=0.5, fliersize=3, color='#D7685F');
```



In [14]:
```
#Histogram for AveBedrms data
sns.histplot(dataset.AveBedrms, color='#D7685F', binwidth=2);
```

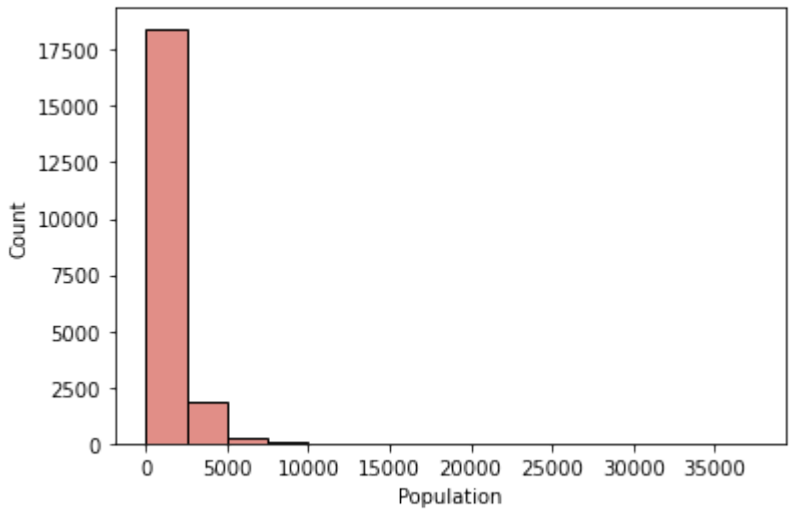


In [15]:
```
#Boxplot for AveBedrms
plt.figure(figsize=(15, 6))
sns.boxplot(x=dataset.AveBedrms, width=0.5, fliersize=3, color='#D7685F');
```

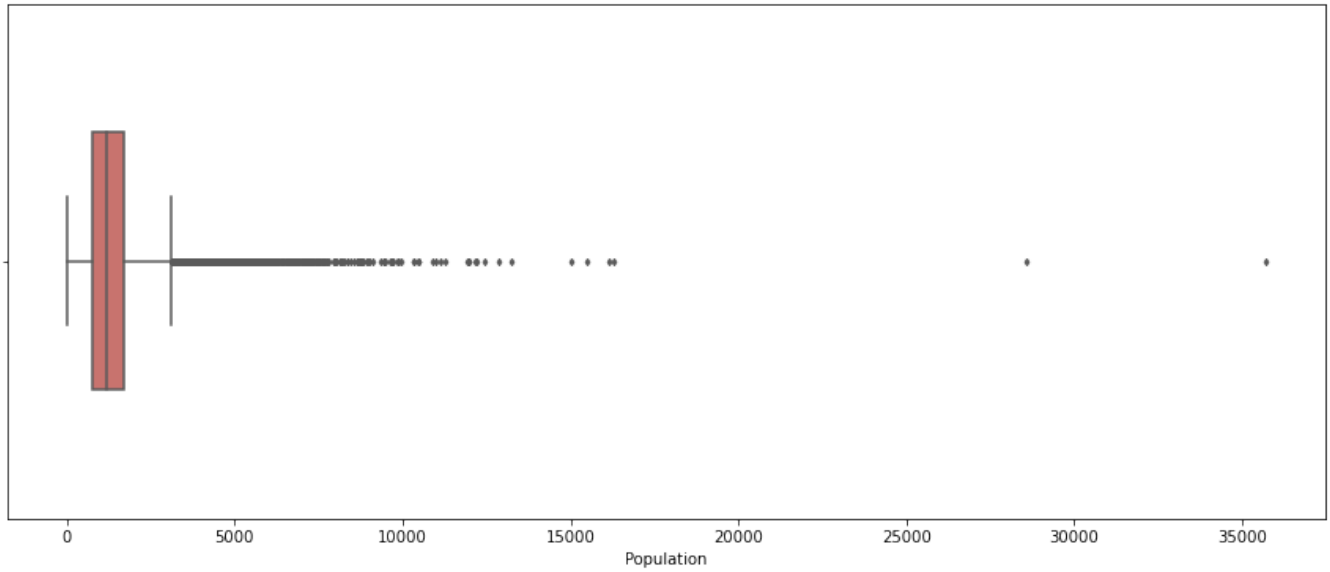


The histogram for average number of rooms and average number of bedrooms is heavily right-skewed due to the presence of outliers.

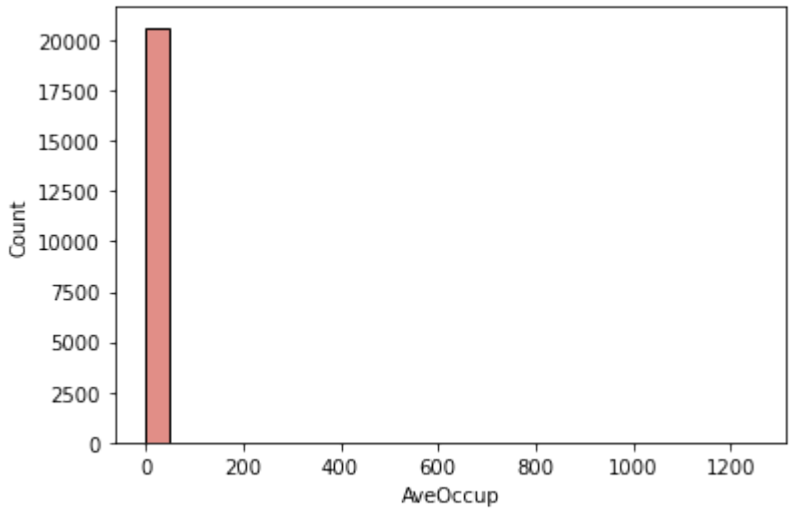
In [16]:
```
#Histogram for Population data
sns.histplot(dataset.Population, color='#D7685F', binwidth=2500);
```



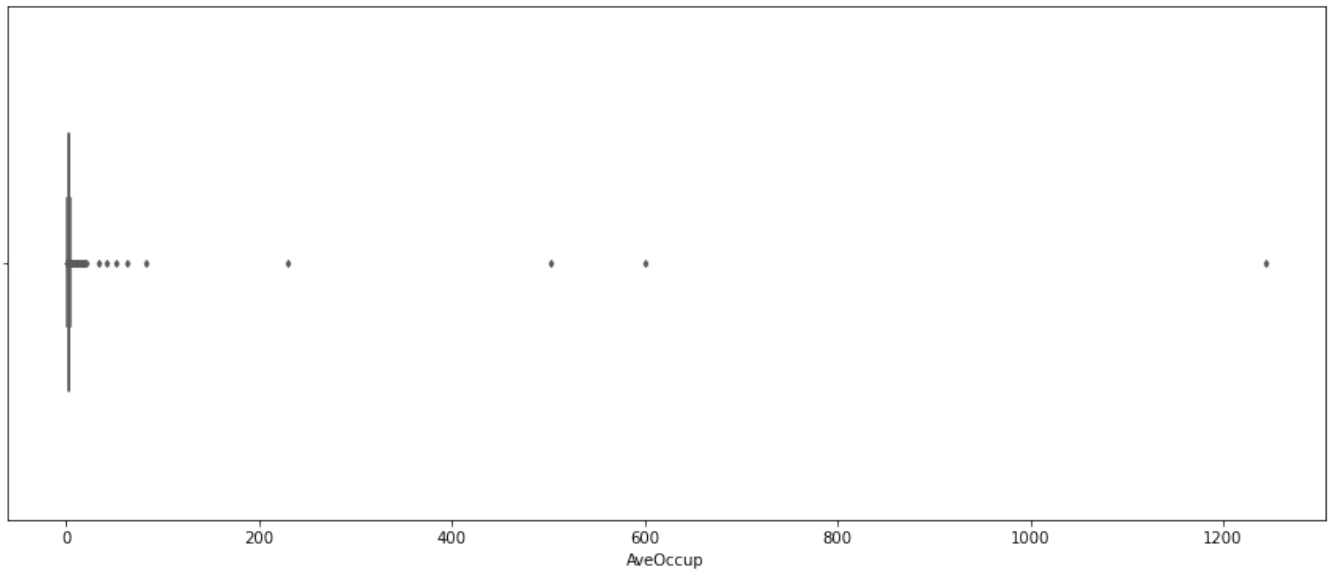
In [17]:
```
#Boxplot for Population data
plt.figure(figsize=(15, 6))
sns.boxplot(x=dataset.Population, width=0.5, fliersize=3, color='#D7685F');
```



In [18]:
```
#Histogram for AveOccup data
sns.histplot(dataset.AveOccup, color='#D7685F', binwidth=50);
```

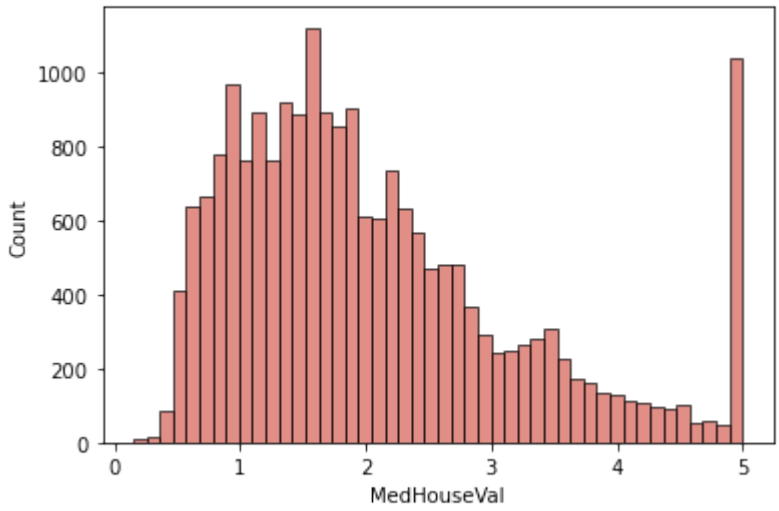


In [19]:
```
#Boxplot for AveOccup
plt.figure(figsize=(15, 6))
sns.boxplot(x=dataset.AveOccup, width=0.5, fliersize=3, color='#D7685F');
```



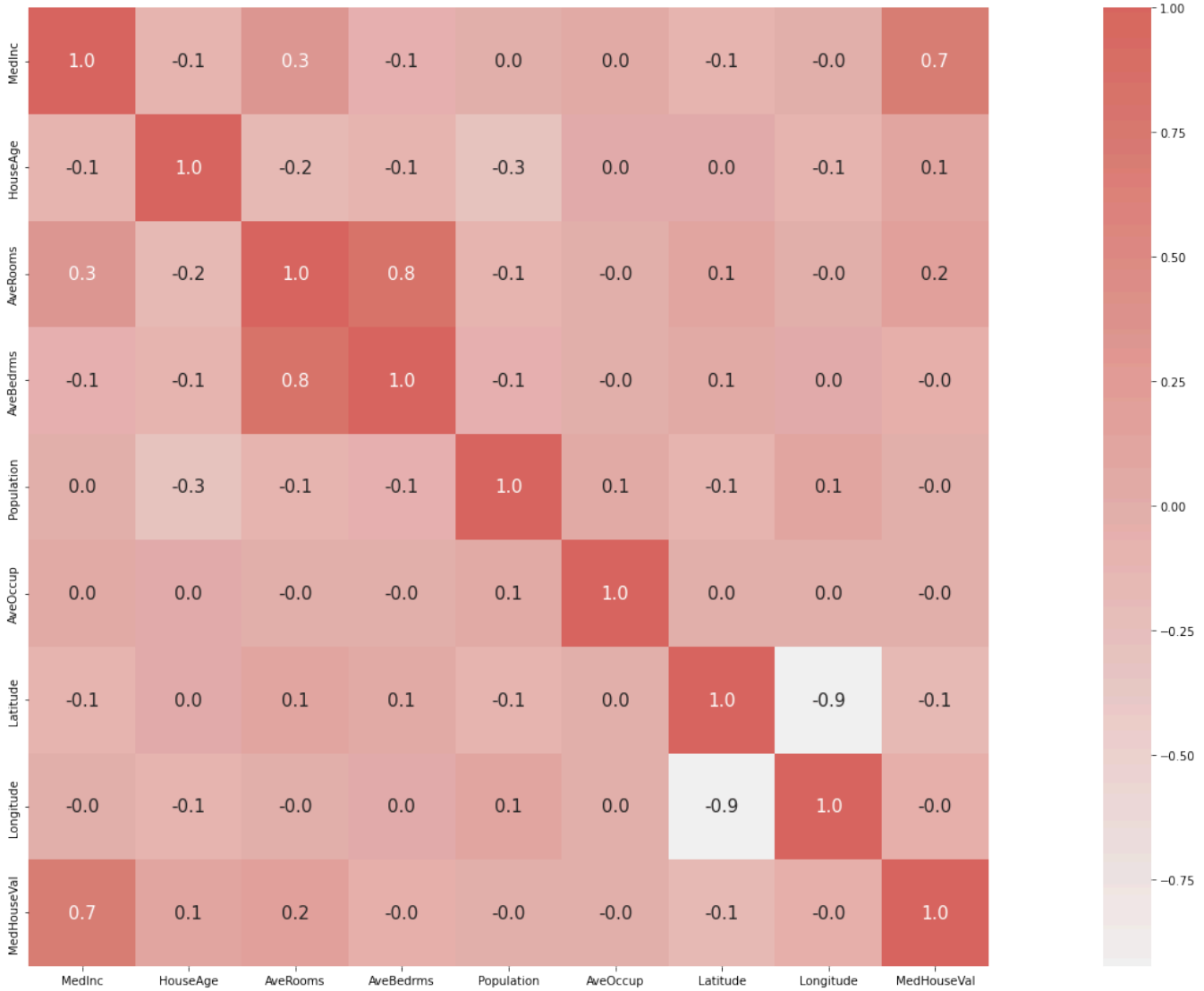
It is evident that the data for Population and Average number of household members contains outliers which need to be dealt with.

In [20]:
```
#Histogram for MedHouseVal data
sns.histplot(dataset.MedHouseVal, color='#D7685F');
```



There aren't any outliers in the MedHouseVal (House Price) since values greater than 5 have been considerd to be equal to 5 which makes the histogram predominantly normally distributed.

In [21]:
```
#Correlation analysis
corr = dataset.corr()
plt.figure(figsize=(35,15))
sns.heatmap(corr, cbar=True, square= True, fmt='.1f', annot=True, annot_kws={'size':15}, cmap
```



In [22]:
```
dataset.shape
```

Out[22]:
```
(20640, 9)
```

There is strong correlation between AveRooms and AveBedrms data. This makes sense since a bigger house is more likely to have more bedrooms.

Hence, removing the data associated with outliers from only one of the two columns would suffice.

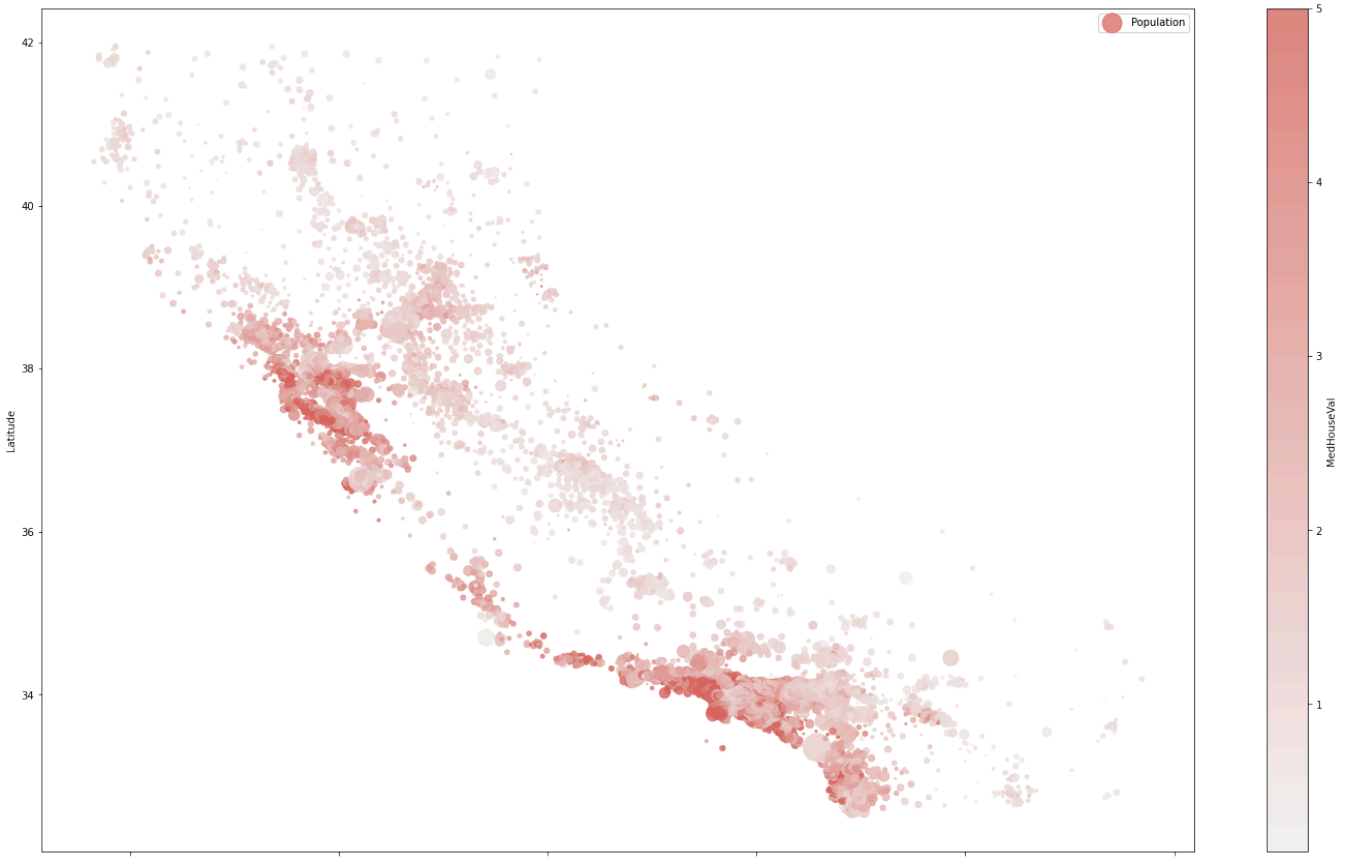
MedInc and MedHouseVal also show a strong correlation. This also makes sense since households with higher income have higher purchasing power.

In [23]:
```
dataset[(dataset['AveRooms'] > 50)]# & (dataset['AveBedrms'] < 20)]
```

Out[23]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
1912	4.9750	16.0	56.269231	10.153846	54.0	2.076923	39.01	-120.16	2.06300
1913	4.0714	19.0	61.812500	11.000000	112.0	2.333333	39.01	-120.06	4.37500
1914	1.8750	33.0	141.909091	25.636364	30.0	2.727273	38.91	-120.10	5.00001
1979	4.6250	34.0	132.533333	34.066667	36.0	2.400000	38.80	-120.08	1.62500
2395	3.8750	23.0	50.837838	10.270270	64.0	1.729730	37.12	-119.34	1.25000
9676	3.2431	14.0	52.848214	11.410714	265.0	2.366071	37.64	-119.02	2.21400
11707	1.1912	22.0	52.690476	8.857143	98.0	2.333333	39.15	-120.06	1.70000
11862	2.6250	25.0	59.875000	15.312500	28.0	1.750000	40.27	-121.25	0.67500
12447	1.6154	17.0	62.422222	14.111111	83.0	1.844444	33.97	-114.49	0.87500

In [59]:
```
#Visualizing Dependent variable against location specific attributes
dataset.plot(kind='scatter', x='Longitude', y='Latitude', alpha=0.8, s=dataset['Population']/
             label='Population', c='MedHouseVal', cmap=sns.light_palette("#D7685F", as_cmap=T
             figsize=(25, 15))
plt.show()
```



Districts with largest values MedHouseVal are in close promiximity to certain locations. This implies that perhaps, these locations share a distinct characteristic such as being closer to downtown for example.

Since we do not have relevant data to test this assumption, we'll treat these values as outliers.

This will ensure that the Machine Learning Model to be built will generalize better to the entire dataset and subsequently to the new unseen data that will be used to make future predictions

---

4. Data Preprocessing

Removing outliers :

In [10]:
```
#Calculate the mean and standard deviation for AveRooms
mean_AveRooms = np.mean(dataset.AveRooms)
sd_AveRooms = np.std(dataset.AveRooms)
mean_AveRooms, sd_AveRooms
```

Out[10]:
```
(5.428999742190365, 2.474113202333518)
```

In [11]:
```
#Identify points that are more than 3 standard deviations away
three_sd=[rooms for rooms in dataset.AveRooms if (rooms < mean_AveRooms - 3*sd_AveRooms) or (
```

In [12]:
```
#Identify points that are more than 4 standard deviations away
four_sd=[rooms for rooms in dataset.AveRooms if (rooms < mean_AveRooms - 4*sd_AveRooms) or (r
```

In [13]:
```
#Identify points that are more than 6 standard deviations away
six_sd=[rooms for rooms in dataset.AveRooms if (rooms < mean_AveRooms - 6*sd_AveRooms) or (ro
```

In [28]:
```
#Dropping outliers from AveRooms
dataset = dataset[~dataset["AveRooms"].isin(six_sd)]
dataset.shape
```

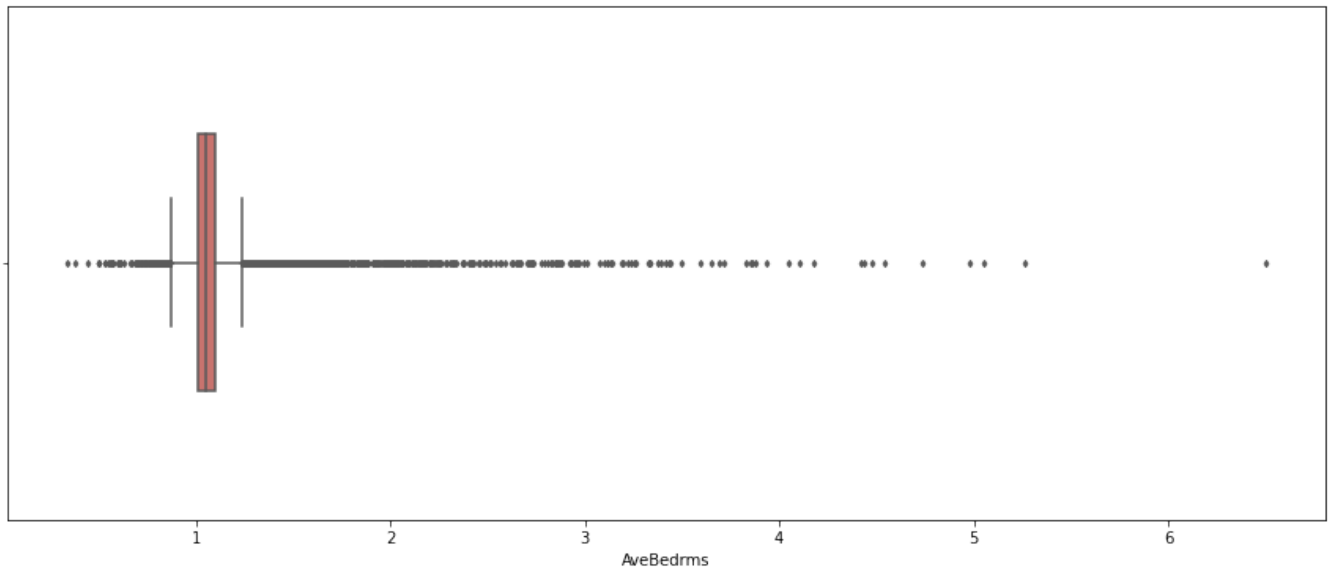
Out[28]:
```
(20575, 9)
```

To better generalize the model that will be built in the later sections- rows where the value of AveRooms that is six standard deviations away from the mean have been dropped.

Given the strong relationship between number of rooms and number of bedrooms in an estate (correlation = 0.8), outliers from AveBedrms should have been automatically resolved in the previous step.

We can confirm this by checking the box plot below

In [29]:
```
#Confirming the removal of outliers from AveBedrms
plt.figure(figsize=(15, 6))
sns.boxplot(x=dataset.AveBedrms, width=0.5, fliersize=3, color='#D7685F');
```



In [30]:
```
#Calculate the mean and standard deviation for Population
mean_Population = np.mean(dataset.Population)
sd_Population = np.std(dataset.Population)
mean_Population, sd_Population
```

Out[30]:
```
(1429.0344106925882, 1132.3152005178606)
```

In [31]:
```
#Identify points that are more than 4 standard deviations away
four_sd_pop=[rooms for rooms in dataset.Population if (rooms < mean_Population - 4*sd_Populat
```

In [32]:
```
#Dropping outliers from Population data
dataset = dataset[~dataset["Population"].isin(four_sd_pop)]
dataset.shape
```

Out[32]:
```
(20380, 9)
```

To better generalize the model that will be built in the later sections- rows where the value of Population that is four standard deviations away from the mean have been dropped.

In [33]:
```
#Calculate the mean and standard deviation for AveOccup
mean_AveOccup = np.mean(dataset.AveOccup)
sd_AveOccup = np.std(dataset.AveOccup)
mean_AveOccup, sd_AveOccup
```

Out[33]:
```
(2.970752754534313, 4.319128270401419)
```

The values of mean and standard deviation for AveOccup have changed from (3.071, 10.386) to (2.972, 4.323) respectively.

Thus, removal of outliers with resepct to the Population data changed the statistical signature of the AveOccup data.

This makes sense considering the AvgOccup shows no linear relationship with any other variables except Population (albeit, it's merely 0.1).

In [34]:
```
#Identify points that are more than 4 standard deviations away
four_sd_occ=[rooms for rooms in dataset.AveOccup if (rooms < mean_AveOccup - 4*sd_AveOccup) o
```

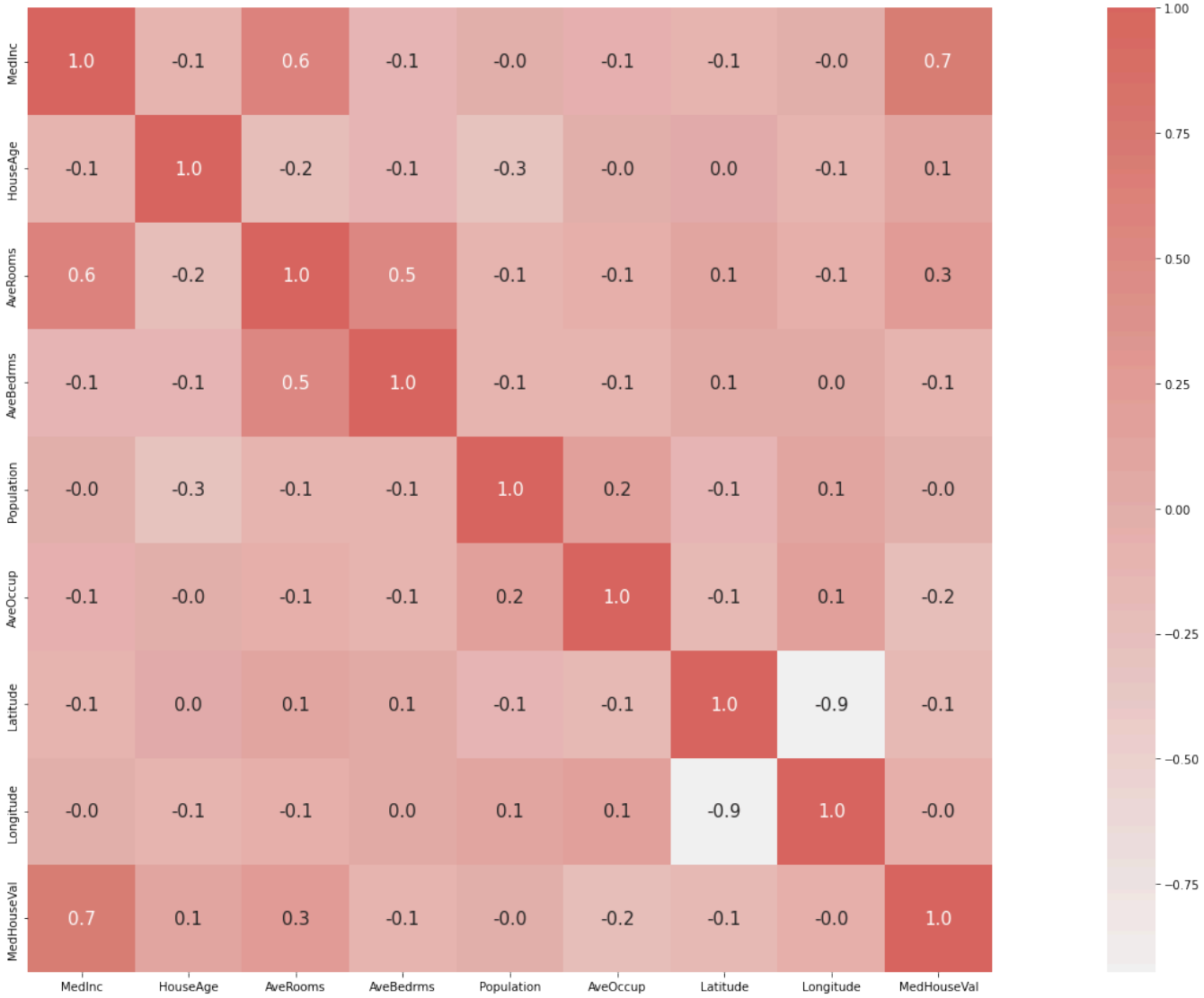
In [35]:
```
#Identify points that are more than 6 standard deviations away
six_sd_occ=[rooms for rooms in dataset.AveOccup if (rooms < mean_AveOccup - 6*sd_AveOccup) or
```

In [36]:
```
#Dropping outliers from AveOccup data
dataset = dataset[~dataset["AveOccup"].isin(six_sd_occ)]
dataset.shape
```

Out[36]:
```
(20375, 9)
```

Rows where the value of AveOccup that is six standard deviations away from the meaa have been dropped.

In [37]:
```
#Correlation analysis for cleaned dataset
corr = dataset.corr()
plt.figure(figsize=(35,15))
sns.heatmap(corr, cbar=True, square= True, fmt='.1f', annot=True, annot_kws={'size':15}, cmap
```



After removing outliers, the data shows a stronger correlation between AveRooms and MedInc.

This suggests that households with a higher income tend to perfer more space than more bedrooms in the property.

This espcially makes sense since the relationship between number of household members and median income of the block is non-existent.

Feature Engineering :

In [38]:
```
#Bedroom to Room ratio
dataset['BedroomRatio'] = dataset['AveBedrms']/dataset['AveRooms']
dataset.head()
dataset.shape
```

Out[38]:
```
(20375, 10)
```

In [39]:
```
#Rearragning columns
dataset = dataset[['MedInc','HouseAge','AveRooms','AveBedrms','Population','AveOccup','Latitu
```

Preparing the data for modelling :

In [40]:
```
#Creating matrix of independent variables
X = pd.DataFrame(data=dataset.iloc[:, :-1])
print("Matrix of features/independent variables: ", X.shape)
```

```
Matrix of features/independent variables:  (20375, 9)
```

In [41]:
```
#Creating vector of dependent variable
y = pd.DataFrame(data=dataset.iloc[:, -1])
print("Vector of dependent variable: ", y.shape)
```

```
Vector of dependent variable:  (20375, 1)
```

In [42]:
```
#Splitting the data into training and test sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 1)
```

Feature Scaling :

In [43]:
```
#Standardisation
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Dimensionality Reduction :

We will forego the application of dimensionality reduction since there are a relatively low number of variables / predictors in the dataset.

---

5. Model Creation

Decision Tree :

In [44]:
```
#Creating object 'dt' of DecisionTreeRegressor
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeRegressor
dt = DecisionTreeRegressor()
```

In [45]:
```
#Applying k-fold cross validation
R2_dt = cross_val_score(estimator=dt, X=X_train, y=y_train, cv=10, scoring='r2')
print("R^2: {:.2f} %".format(R2_dt.mean()*100))
print("Standard Deviation: {:.2f} %".format(R2_dt.std()*100))
```

```
R^2: 57.85 %
Standard Deviation: 3.96 %
```

Random Forest :

In [46]:
```
#Creating object 'rf' of RandomForestRegressor
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(n_estimators = 100, random_state = 0)
```

In [47]:
```
#Applying k-fold cross validation
from sklearn.model_selection import cross_val_score
R2_rf = cross_val_score(estimator=rf, X=X_train, y=y_train.values.ravel(), cv=10, scoring='r2
print("R^2: {:.2f} %".format(R2_rf.mean()*100))
print("Standard Deviation: {:.2f} %".format(R2_rf.std()*100))
```

```
R^2: 80.18 %
Standard Deviation: 1.44 %
```

XGBoost :

In [48]:
```
#Creating object 'xgb' of XGBRegressor
from xgboost import XGBRegressor
xgb = XGBRegressor()
```

In [49]:
```
#Applying k-fold cross validation
R2_xgb = cross_val_score(estimator=xgb, X=X_train, y=y_train, cv=10, scoring='r2')
print("R^2: {:.2f} %".format(R2_xgb.mean()*100))
print("Standard Deviation: {:.2f} %".format(R2_xgb.std()*100))
```

```
R^2: 83.38 %
Standard Deviation: 1.21 %
```

---

6. Model Evaluation

In [50]:
```
#Training XGBoost on the Training set
xgb.fit(X_train, y_train)
```

Out[50]:
```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             gamma=None, grow_policy=None, importance_type=None,
             interaction_constraints=None, learning_rate=None, max_bin=None,
             max_cat_threshold=None, max_cat_to_onehot=None,
             max_delta_step=None, max_depth=None, max_leaves=None,
             min_child_weight=None, missing=nan, monotone_constraints=None,
             multi_strategy=None, n_estimators=None, n_jobs=None,
             num_parallel_tree=None, random_state=None, ...)
```

In [51]:
```
#Predicting the Test set results
y_pred = xgb.predict(X_test)
```

In [52]:
```
#Evaluating model performance
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

Out[52]:
```
0.84765575953374
```

Since there isn't a huge difference in the R2 scores for training and test set, it implies that the model is not overfitting the training data (and is generalizing well to the unseen data in the test set). We will therefore forego the implementation of regularization techniques that are used to address model overfitting.

We can thus conclude that XGBoost performs the best out of all the 3 models and hence will be used for API creation and containerising the model using Docker

Saving model artificats for deployment purposes :

In [53]:
```
pip show xgboost
```

```
Name: xgboost
Version: 2.0.3
Summary: XGBoost Python Package
Home-page:
Author:
Author-email: Hyunsu Cho <chohyu01@cs.washington.edu>, Jiaming Yuan <jm.yuan@outlook.com>
License: Apache-2.0
Location: c:\users\aashi\anaconda3\lib\site-packages
Requires: numpy, scipy
Required-by:
Note: you may need to restart the kernel to use updated packages.
```

In [54]:
```
#Pickling necessary python objects
import pickle
```

In [55]:
```
#Pickling the XGBoost model
with open(r'C:\Users\aashi\OneDrive\Desktop\Portfolio\Analysis and Prediction of House Prices
    pickle.dump(xgb, model_pkl)
```

In [56]:
```
#Pickling the StandardScaler object 'sc'
with open(r'C:\Users\aashi\OneDrive\Desktop\Portfolio\Analysis and Prediction of House Prices
    pickle.dump(sc, sc_pkl)
```

XGBoost Model: Technical Overview

Introduction:

XGBoost belongs to the Gradient Boosting class of Machine Learning models that rely on the decision tree framework. It is arguably one of the most popular machine learning models for Regression and Classification based modelling due to its high efficiency. XGBoost model was designed to be used with large, complicated datasets.

For ease of understanding and to better internalize the model however, the explanation will be based on a dataset with 4 observations, containing 1 independent variable and 1 dependent variable.

Data: The data points below are directly taken from the California Housing Market dataset –

AveRooms	MedHouseVal
2.7	0.6
5.5	2.8
6.9	4.5
3.8	1.3

Here, AveRooms is the average number of rooms per household (independent variable) and
MedHouseVal is the Median Property Valuation in $100,000 (dependent variable).

Implementing the XGBoost Model for Regression:

1. Initial Predictions:
    a. The first step to fit the XGBoost model to the training set is to make an initial prediction.
    b. A prediction value of 0.5 is chosen by default

AveRooms	MedHouseVal	Prediction

2.7	0.6	0.5
5.5	2.8	0.5
6.9	4.5	0.5
3.8	1.3	0.5

2. Calculating Pseudo Residuals:
    a. Residuals are the differences between Actual Values and the Predicted Values
    b. XGBoost fits a regression tree to the residuals

AveRooms	MedHouseVal	Prediction	Psuedo Residuals
2.7	0.6	0.5	0.1
5.5	2.8	0.5	2.3
6.9	4.5	0.5	4
3.8	1.3	0.5	0.8

3. Building XGBoost Trees:
    a. Each tree starts out as single leaf. All residuals go under this leaf. 0.1, 2.3, 4, 0.8
    b. We now calculate a quality score known as a Similarity Score for the residuals

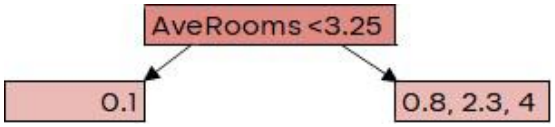
$$Similarity\ Score = \frac{(Sum\ of\ Residuals)^2}{Number\ of\ Residuals + \lambda}$$

Here, λ is the regularization parameter, which we'll set to 0 for now.

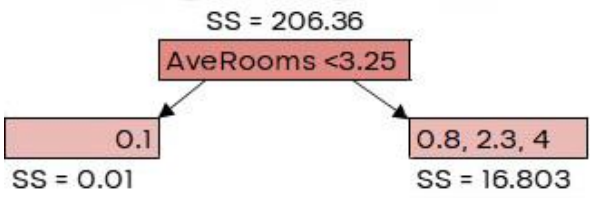
Similarity Score = 206.36

    c. Improving Similarity Score:
        i. We can improve the Similarity Score by creating new leaves.
        ii. Criteria for creating the first new leaf:
            1. Sort the observations for average rooms in ascending order.
            2. Calculate the average of the 1st two observations = 3.25

3. Split all Psuedo Residuals into 2 new leaves based on the threshold AveRooms < 3.25

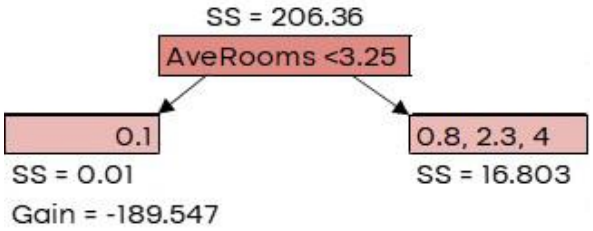


4. Calculate Similarity Score (SS) for the two new leaves



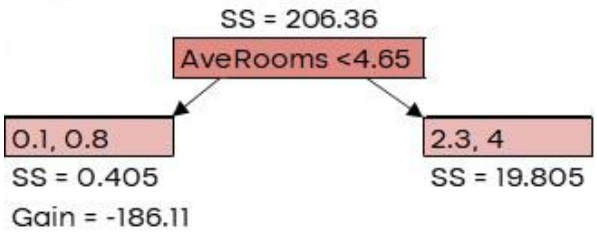
5. Calculate the Gain: We will now determine the degree to which the leaves cluster similar residuals when compared to the root.

$$Gain = Left\ leaf_{SS} + Right\ leaf_{SS} - Root_{SS}$$



iii. Criteria for creating second new leaf:

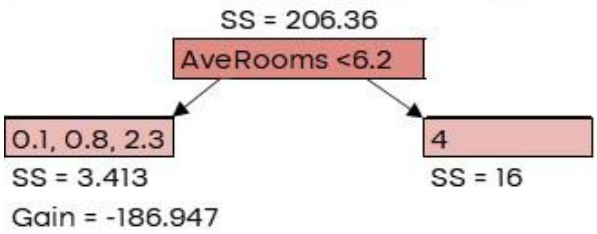
1. Sort the observations for average rooms in ascending order.
2. Calculate the average of the 2nd two observations = 4.65
3. Split all Psuedo Residuals into 2 new leaves based on the threshold AveRooms < 4.65
4. Calculate SS for the two new leaves
5. Calculate Gain for this new threshold



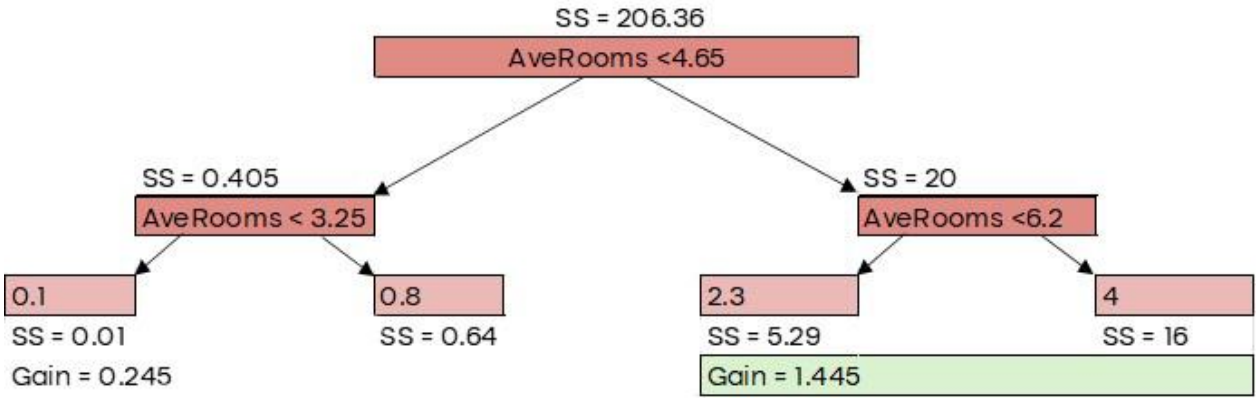
iv. Criteria for creating third new leaf:

1. Sort the observations for average rooms in ascending order.
2. Calculate the average of the last two observations = 4.65

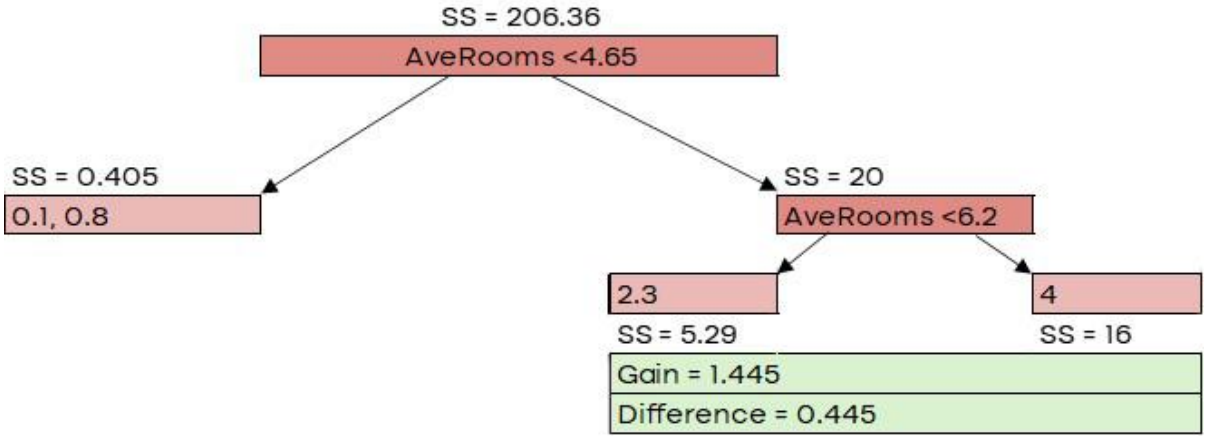
3. Split all Psuedo Residuals into 2 new leaves based on the threshold AveRooms < 6.2
4. Calculate SS for the two new leaves
5. Calculate Gain for this new threshold



v. Split the tree based on threshold with the largest gain (i.e. AveRooms < 4.65)
vi. Creating first new branch:
    1. Create new branches from the leaves for this threshold
    2. Calculate SS and Gain for new leaves and new thresholds
    3. Select the branch with the higher gain to split the tree
    4. Note that the branch creation is based on the tree depth. Here Tree depth = 3. The default is to allow up to 6 levels



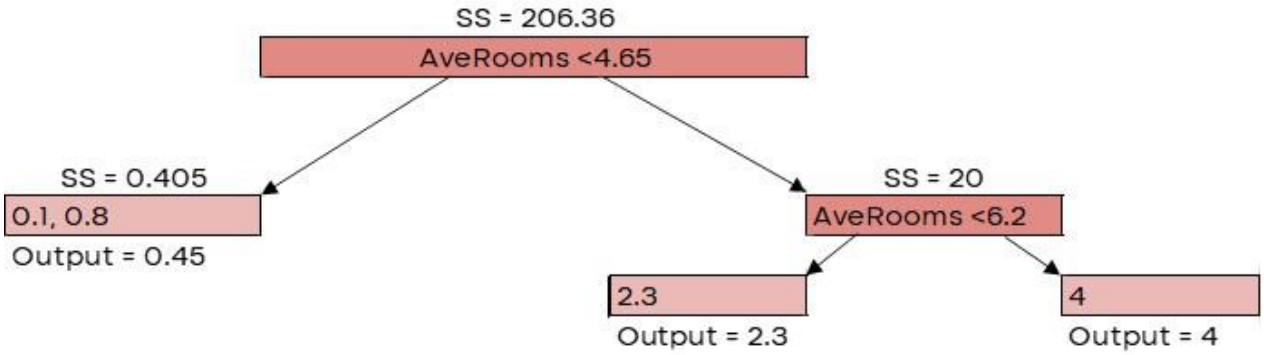
vii. Pruning the Tree:
    1. We use Gain values and gamma (γ) to prune the tree. Note that larger the γ, more extreme is the tree pruning. Here, $\gamma = 1$.
    2. we calculate difference between the gain associated with the lowest branch in the tree and gamma
    3. if this difference is negative, we'll remove the branch. if it's positive, we'll not remove the branch
    4. Here the difference is 1.445 - 1 = 0.445 which is positive. Hence, we will retain this branch



viii. Calculating output value:

1. We'll now calculate Output values for the new leaves by using the below formula -

$$Output\ Value = \frac{Sum\ of\ Residuals}{Number\ of\ Residuals + \lambda}$$



ix. Calculating New Predictions (and new Pseudo Residuals):

1. To calculate the new predictions, we will use a learning rate of 0.3. Note that learning rate is like a knob that can be adjusted to move towards the correct predictions in increments. The smaller the value, the slower the algorithm converges to the correct predictions.
2. $New\ Predictions\ = Old\ predictions + (Learning\ rate * Output)$

AveRooms	MedHouseVal	New Prediction
2.7	0.6	0.635
5.5	2.8	1.19
6.9	4.5	1.7
3.8	1.3	0.635

3. Now we calculate the new pseudo residuals

AveRooms	MedHouseVal	New Predictions	New Psuedo Residuals
2.7	0.6	0.635	-0.035
5.5	2.8	1.19	1.61
6.9	4.5	1.7	2.8
3.8	1.3	0.635	0.665

We can see that these new residuals are smaller than the old ones. Hence, we've taken a small step in the right direction.

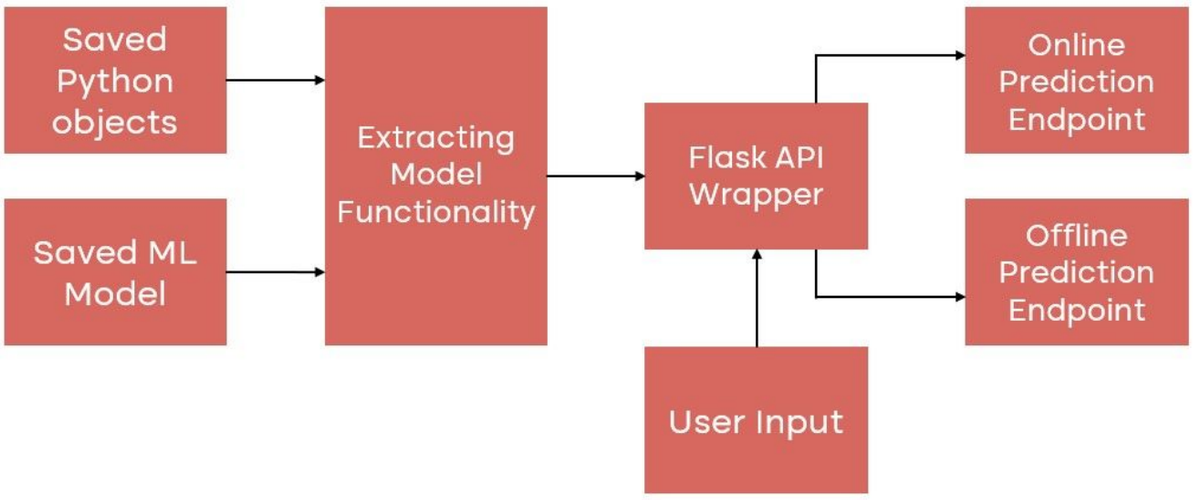
d. We keep building new trees that give us smaller and smaller residuals until residuals are very small or we've reached the max number.
e. Increasing the value of λ for new trees:
    i. Remember that λ is a regularization parameter. It reduces the sensitivity an individual observation has on the prediction values
    ii. When $\lambda > 0$, the SS is smaller. The amount of decrease is inversely proportional to the number of residuals in the node.
    iii. Note that it's easier to prune leaves when $\lambda > 0$ because gain values are smaller and the whole point of λ is to prevent overfitting the training data
    iv. I.e. when $\lambda > 0$ it will reduce the impact of the individual observation on the overall prediction

API Endpoint Integration

Overview:

The API for our best performing Machine Learning Model is built to accept a CSV file from the user in order to generate predictions for property valuation for each entry in the file. The results are then made available to the user as a downloadeable file for ease of access.

This API can also be customized for additional endpoints based on specific user requirements. For example: entering attributes for predicting the price of an individual property instead of uploading a CSV file.



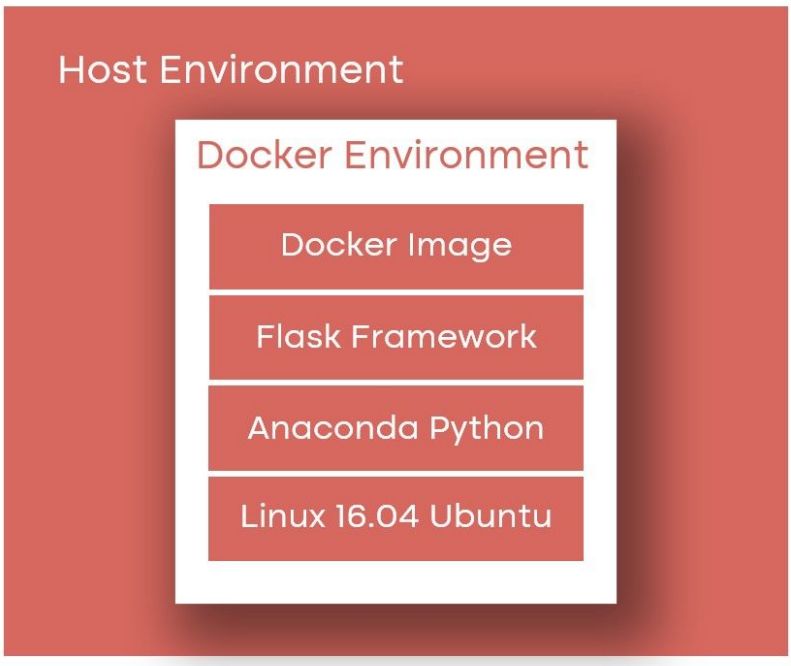
```
@author: aashi
"""
import io
import pickle
from flask import Flask, request, send_file
from flasgger import Swagger
import pandas as pd


#Loading the trained XGBoost model using pickle
with open(r'../analysis_and_prediction_of_house_prices/model_artifacts/xgb.pkl', 'rb') as moc
    model = pickle.load(model_pkl)


#Loading the saved scaler using pickle
with open(r'../analysis_and_prediction_of_house_prices/model_artifacts/sc.pkl', 'rb') as sc_p
    sc = pickle.load(sc_pkl)


app = Flask(__name__)
swagger = Swagger(app)


#Defining function to create a new column - BedroomRatio
def calculate_bedroom_ratio(dataset):
    dataset['BedroomRatio'] = dataset['AveBedrms'] / dataset['AveRooms']
    return dataset


#Defining function to rearrange specific columns
def rearrange_columns(dataset):
    # Define the columns you want to keep and rearrange
    expected_columns = ['MedInc','HouseAge','AveRooms','AveBedrms','Population','AveOccup','I
    # Select only the available columns in the dataset
    dataset = dataset[[col for col in expected_columns if col in dataset.columns]]
    return dataset


# Defining API endpoint to handle file upload
@app.route('/predict', methods=['POST'])
def predict():
    """
```

```
    This is the prediction endpoint.
    It takes a CSV file as input and returns predictions.
    ---
    parameters:
      - name: input_file
        in: formData
        type: file
        required: true
        description: The CSV file containing the input data.
    responses:
      200:
        description: Predictions of Property Valuation
    """
    #reading the input file
    input_data = pd.read_csv(request.files.get("input_file"))

    #retaining the original input data
    original_input_data = input_data

    #creating new column i.e. feature engineering
    input_data = calculate_bedroom_ratio(input_data)

    #rearraging columns to match expected order for model consumption
    input_data = rearrange_columns(input_data)


    #standardizing numerical variables using saved scaler
    input_data = sc.transform(input_data)

    #performing predictions using the pre-trained model
    predictions = model.predict(input_data)

    #appending predictions to the original input data
    original_input_data['Predictions_MedHouseVal'] = predictions


    #creating an in-memory CSV file with both input data and predictions
    output = io.BytesIO()
    original_input_data.to_csv(output, index=False, encoding='utf-8')
    output.seek(0)

    #sending the CSV file as a response
    return send_file(output, mimetype='text/csv', as_attachment=True, attachment_filename='p


if __name__ == '__main__':
    app.run(host='0.0.0.0', port = 5000, debug=True)
```

Creating Docker file

Overview:

The Machine Learning API is then packaged into a Docker file which is used for containerzing the Model. This ensures that Model is standardized and the results are reproducible and replicable across different environments and machines.



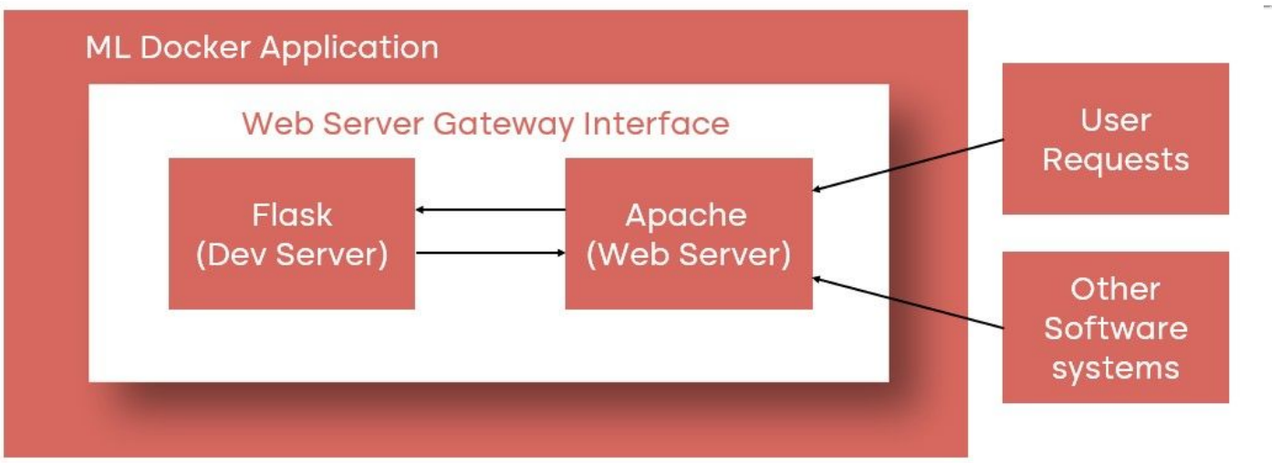
```
FROM continuumio/anaconda3:2021.05

EXPOSE 8000

RUN apt-get update && \
    apt-get install -y apache2 \
    apache2-dev \
    vim \
 && apt-get clean \
 && apt-get autoremove \
 && rm -rf /var/lib/apt/lists/*

WORKDIR /var/www/analysis_and_prediction_of_house_prices/

COPY ./ /var/www/analysis_and_prediction_of_house_prices/

RUN pip install -r requirements.txt

RUN /opt/conda/bin/mod_wsgi-express install-module

RUN mod_wsgi-express setup-server "/var/www/analysis_and_prediction_of_house_prices/source_co
    --user www-data --group www-data \
    --server-root=/etc/mod_wsgi-express-80

CMD ["/etc/mod_wsgi-express-80/apachectl", "start", "-D", "FOREGROUND"]
```

Configuring Apache Web Server

Overview:

In order to prepare the Model for deployment on both other local hosts and cloud servers, the Docker file created above is configured such that the Machine Learning API interfaces with a web server (Apache). This is done to ensure that all user requests made to the application are handled efficiently by leveraging the full capabilites and infrastructure offered by a web server.

Note that the Docker file can be customized such that serverless configurations are also available.



```
#! /usr/bin/python

import sys
sys.path.insert(0, "/var/www/analysis_and_prediction_of_house_prices")
sys.path.insert(0,'/opt/conda/lib/python3.6/site-packages')
sys.path.insert(0, "/opt/conda/bin/")

import os
os.environ['PYTHONPATH'] = '/opt/conda/bin/python'

from source_code.analysis_and_prediction_of_house_prices import app as application
```

Model Functionality Showcase

Overview:

The user or a third party can leverage the full capabilities of the Machine Learning Model and start generating predictions in real time.

A swagger API 0.0.1

/apispec_1.json

powered by Flasgger

Terms of service

default

POST `/predict` This is the prediction endpoint. `post_predict`

It takes a CSV file as input and returns predictions.

Parameters Cancel

Name	Description
input_file * required `file` *(formData)*	The CSV file containing the input data. Choose File pr…sv

Execute Clear

Responses Response content type application/json

Curl

```
curl -X POST "http://localhost:5000/predict" -H "accept: application/json" -H "Content-Type: multipart/form-data" -F "input_file=@production test set.csv;type=text/csv"
```

Request URL

```
http://localhost:5000/predict
```

Server response

Code	Details

200	**Response body**
	Download file
	Response headers
	`cache-control: no-cache`
	`connection: close`
	`content-disposition: attachment; filename=predictions_output.csv`
	`content-length: 1890`
	`content-type: text/csv; charset=utf-8`
	`date: Tue29 Oct 2024 10:37:03 GMT`

Code	Details
	server: Werkzeug/3.0.4 Python/3.10.7

Responses

Code	Description
200	Predictions of house prices

[Powered by Flasgger 0.9.7.1]

Framework Deep Dive

1. Exploratory Data Analysis (EDA)
    - Data Sanity Checks
        - Identifying Missing data
        - Identifying Duplicate Data
        - Identifying Inconsistent text/typos/Datatypes
        - Identifying Class Imbalances
    - Filtering, Sorting, Grouping
    - Data Visualization
        - Bar plots, Dot plots
        - Line graphs
        - Pair plots, Scatter Plots
        - Checking data distributions
        - Correlation Analysis
            - Outlier Detection
            - Preliminary Insights

2. Data Preprocessing
    a. Conversions
        i. Converting object columns into datetime columns
        ii. Converting object columns to numeric columns
    b. Handling Missing Data
        i. Data Elimination
        ii. Imputing missing values with mean/median
        iii. Manual imputation based on domain expertise
    c. Handling Inconsistent text/typos
        i. Categorical data, Numerical data
            1. Implementing logical conditions to update values
        ii. Text Data
            1. Stripping unwanted characters
            2. Turning text to lowercase
    d. Deleting Duplicate Data

e. Handling Outliers
    i. Removing Outliers based on Standard Deviation
    ii. Resolving the issue based on domain expertise
f. Feature Engineering
    i. Creating new columns based on domain expertise and EDA findings
    ii. Appending, Joining
    iii. Categorical Data Transformation
        1. One-hot encoding
        2. Label Encoding
    iv. Numerical Data Transformation
        1. Standardization
        2. Normalization
    v. Dimensionality Reduction
        1. Principal Component Analysis (PCA)
        2. Linear Discriminant Analysis (LDA)
g. Handling Class imbalances
    i. Stratifying Response Variables

3. Machine Learning Models
    a. Regression
        i. Simple Linear Regression
        ii. Multiple Linear Regression
        iii. Polynomial Regression
        iv. Support Vector Regression (SVR)
        v. Decision Tree Regression
        vi. Random Forest Regression
        vii. XGBoost
    b. Classification
        i. Logistic Regression
        ii. K-Nearest Neighbors (K-NN)
        iii. Support Vector Machine (SVM)
        iv. Kernel SVM
        v. Naive Bayes
        vi. Decision Tree Classification

vii. Random Forest Classification
viii. XGBoost

c. Clustering
    i. K-Means Clustering
    ii. Hierarchical Clustering

d. Association Rule Learning
    i. Apriori
    ii. Eclat

e. Natural Language Processing
    i. Term Frequency and Inverse Document Frequency (Tf-IDf) Model
    ii. Bag Of Words (BOW) Model

4. Model Boosting
    a. Regularization: Ridge and Lasso
    b. k-Fold Cross Validation
    c. Grid Search
    d. Hyperparameter Tuning

5. Model Evaluation metrics
    a. R-squared, Adjusted R-squared
    b. MSE (Means Square Error), MAE (Mean Absolute Error), RMSE (Root Mean Square Error)
    c. Confusion Matrix
    d. Accuracy, Precision, Recall
    e. Standard Deviation, Variance

6. Creating Machine Learning Model Artifacts
    a. Capture version dependencies
    b. Creating pickle files in Python
        i. Saving feature transformers used for model training
        ii. Extracting best performing model from research environment

7. API Creation
    a. Configure the model for API integration

        i. Load the previously saved pickle files for models, other python objects
        ii. Write the code to convert the Machine Learning model into an API
    b. Set up user interface for developers/end users
        i. Implementing Swagger UI from Flasgger Flask Module
    c. Expose model functionality as Flask API endpoints
        i. Single instance prediction endpoint
        ii. Multi-instance prediction endpoint

8. Containerizing the Model
    a. Create Docker file for local deployment
        i. Set up Docker on local machine
        ii. Execute necessary docker commands in windows PowerShell
        iii. Build the docker image
    b. Testing/Debugging the model on local machine/development server
        i. Run the docker image and verify API response

9. Configuring Web Server
    a. Configure Web Server runtime environment/software dependencies
        i. Specify file paths
        ii. Specify package versions
        iii. Specify OS
    b. Integrate local Flask API with Apache Web Server (WSGI)
        i. Create a Web Server Gateway Interface file
    c. Instantiate Flask API as a web application

10. Operationalizing the Model as a Microservice
    a. Re-create docker file for web server deployment
        i. Add docker commands to enable WSGI functionality
    b. Ensure model scalability and reproducibility across machines
        i. Ensure package versions and software dependencies across research and production environments are used
    c. Deploy and Capture model response with production/test dataset in real time

i. Build the updated docker image
ii. Run the updated docker image to create a new container

d. Testing/debugging on production server
    i. Verify if the docker container is running successfully
    ii. Connect to Apache web server error logs for debugging
    iii. Confirm model response post testing